

MetaKLController — Critical Review and Revised Implementation Proposal

Reviewer: Claude (Anthropic)
Document reviewed: MetaKL Implementation Proposal v1
Repository: ruth22soft/RL-math-reasoner, branch: v1
Date: 2026-04-18
Status: Revised proposal with corrections

Part 1 — Critical Review of Original Proposal

1.1 What is good (keep as-is)

Section	Verdict	Reason
Phase A → Phase B staged approach	Keep	De-risks integration. Correct engineering practice.
Separating actor KL from trainer KL	Keep	Critical distinction. Proposal correctly identifies that <code>algorithm.kl_ctrl</code> does not affect actor KL when <code>use_kl_loss=True</code> .
State vector <code>[kl_loss, reward_mean, reward_std, grad_norm]</code>	Keep	Well-motivated from training signals. Consistent with thesis algorithm.
Running mean/std normalization with clip <code>[-5, 5]</code>	Keep	Prevents LSTM from seeing wildly scaled inputs. Essential.
Backward-compatible config switch	Keep	<code>actor_adaptive_kl.enable: false</code> preserves baseline behavior.
Ablation matrix (Section 9)	Keep	Four-condition ablation strengthens thesis considerably.
Logging diagnostics (Section 8)	Keep	Comprehensive. Every logged metric has a clear diagnostic purpose.

1.2 Critical errors that must be fixed

Error 1 — CRITICAL: Wrong baseline `kl_loss_coef` in Section 6.1

Original proposal says:

```
yaml

actor_rollout_ref.actor.kl_loss_coef: 0.001
```

Your actual run_training.sh uses:

```
yaml

actor_rollout_ref.actor.kl_loss_coef: 0.0001    # 10x smaller
```

Why this matters:

Your entire thesis is built around the observation that $\beta = 1 \times 10^{-4}$ produced a flat KL loss at 0.001. If Phase A initialises at $\beta = 0.001$ (10× higher), you are not comparing against your documented baseline — you are running a completely different experiment. Your advisor will immediately catch this when comparing Section 6.1 to your baseline report.

Fix: Change `(init_beta)` to `(0.0001)` everywhere. Your baseline, Phase A, and Phase B must all start from the same $\beta_0 = 1 \times 10^{-4}$.

Error 2 — CRITICAL: `(target_kl_loss = 0.08)` is 80× too high

Original proposal says:

```
yaml

actor_adaptive_kl.target_kl_loss: 0.08
```

Your actual training logs show:

```
KL loss remained consistently near 0.001 throughout all 444 steps
```

Why this matters:

The rule-based controller increases β when `(kl_loss > target_kl_loss)`. If the target is 0.08 but your actual KL loss runs at 0.001, the condition `(kl_loss > 0.08)` will NEVER be true. The controller will perpetually decrease β toward `(min_beta)` and stay there. Phase A will produce a lower β than baseline — not an adaptive one. This completely invalidates Phase A as an ablation.

Fix: Set `(target_kl_loss: 0.003)` (3× your observed 0.001, leaving reasonable headroom for genuine increases before the controller reacts).

Error 3 — SIGNIFICANT: `(warmup_steps: 150)` is 34% of your entire epoch

Original proposal says:

```
yaml
```

```
actor_adaptive_kl.warmup_steps: 150
```

Your epoch has:

444 steps per epoch

Why this matters:

For 150 steps (one third of the entire training run), the controller does nothing and uses the fixed `init_beta`. This destroys the LSTM's ability to learn meaningful early-phase dynamics — which is precisely the motivation for MetaKLController. If your thesis argument is that the controller should adapt during early exploration, making it silent for the first 150 steps contradicts your own contribution.

Fix: Set `warmup_steps: 30` (about one week of steps, enough for the running normalization statistics to stabilise without losing a third of training).

Error 4 — SIGNIFICANT: LSTM gradient flow not addressed

Original proposal describes the LSTM forward pass but never states:

- Whether LSTM weights are updated by gradient backprop through `beta_t * kl_loss`
- Or whether the LSTM is treated as frozen/separate

Why this matters:

This is the fundamental training question for your thesis. If gradients do NOT flow into the LSTM, the controller never learns. If gradients DO flow, the LSTM learns jointly with the policy. The proposal has no position on this — which means it could be implemented either way by accident, producing irreproducible results.

Fix: Explicitly state in the implementation: gradients flow through `beta_t` into the LSTM via `loss = policy_loss + beta_t * kl_loss`. LSTM weights are updated by the same optimiser step as the actor. This is the joint training approach and requires no separate training loop.

Error 5 — MODERATE: Inconsistent EMA alpha between Phase A and Phase B

Original proposal says:

```
yaml
```

```
Phase A: ema_alpha: 0.95
```

```
Phase B: ema_alpha: 0.90
```

Why this matters:

EMA alpha controls how much history is retained. 0.95 means β changes very slowly (heavy smoothing). 0.90 means β changes faster. If Phase A uses 0.95 and Phase B uses 0.90, any difference in results could be attributed to the EMA difference rather than the LSTM — this confounds your ablation. The ablation should isolate only the controller type.

Fix: Use `ema_alpha: 0.90` for both phases.

Error 6 — MODERATE: `update_interval: 5` for Phase A vs `1` for Phase B

Same confounding problem as Error 5. Phase A updates β every 5 steps, Phase B every step. If Phase B outperforms Phase A, is it because of the LSTM or because it updates more frequently? Your ablation cannot answer this.

Fix: Set `update_interval: 1` for both phases.

Error 7 — MODERATE: No mention of LSTM hidden state reset policy

Original proposal defines the LSTM architecture but never says:

└ When should `(h, c)` be reset to zeros?

Options and their implications:

- Reset every epoch: LSTM starts fresh each epoch. Clean but loses cross-epoch memory.
- Reset every step: Equivalent to a stateless MLP. Destroys the entire LSTM advantage.
- Never reset: LSTM accumulates memory across epochs. Richest signal but harder to train.

Fix: Explicitly state: reset `(h, c)` to zeros at the start of each epoch, not each step. This is the correct choice — the LSTM builds up memory across 444 steps per epoch, which is exactly the temporal context it needs to reason about training dynamics.

Error 8 — MINOR: `ray_trainer.py` modification is riskier than `dp_actor.py`

The proposal modifies `ray_trainer.py` (the outer training loop) to compute `beta_t` and inject it via `meta_info`. This is correct architecturally but introduces a risk: if Ray serialises the batch differently than expected, `meta_info` may not arrive at the actor worker intact.

A safer alternative (also noted in the proposal's `dp_actor.py` section) is to instantiate the controller inside `dp_actor.py` directly, so the LSTM lives on the same worker process that computes the loss. This eliminates the cross-worker serialisation risk.

1.3 Summary of errors

#	Severity	Issue	Impact
1	Critical	Wrong baseline β (0.001 vs 0.0001)	Invalidates comparison with your thesis baseline
2	Critical	target_kl_loss = 0.08 (80× too high)	Phase A controller never activates — always decreases β
3	Significant	warmup_steps = 150 (34% of epoch)	Controller silent during most important early phase
4	Significant	LSTM gradient flow not specified	Implementation could accidentally freeze LSTM weights
5	Moderate	EMA alpha inconsistency (0.95 vs 0.90)	Confounds Phase A vs Phase B ablation
6	Moderate	update_interval inconsistency (5 vs 1)	Same confounding issue
7	Moderate	LSTM hidden state reset policy missing	Could produce irreproducible results
8	Minor	ray_trainer.py injection riskier than dp_actor.py	Potential serialisation failure under Ray

Part 2 — Revised Implementation Proposal

2.1 Corrected parameters

Baseline (your confirmed working config)

```
yaml
actor_rollout_ref.actor.use_kl_loss: true
actor_rollout_ref.actor.kl_loss_coef: 0.0001      # confirmed from run_training.sh
actor_rollout_ref.actor.kl_loss_type: low_var_kl
algorithm.adv_estimator: grpo
```

Phase A — corrected rule-based adaptive

```
yaml
```

```

actor_adaptive_kl.enable: true
actor_adaptive_kl.mode: rule
actor_adaptive_kl.init_beta: 0.0001          # FIXED: was 0.001
actor_adaptive_kl.min_beta: 0.00001
actor_adaptive_kl.max_beta: 0.005
actor_adaptive_kl.warmup_steps: 30           # FIXED: was 150
actor_adaptive_kl.update_interval: 1         # FIXED: was 5
actor_adaptive_kl.ema_alpha: 0.90           # FIXED: was 0.95
actor_adaptive_kl.target_kl_loss: 0.003     # FIXED: was 0.08
actor_adaptive_kl.up_gain: 1.08
actor_adaptive_kl.down_gain: 0.94
actor_adaptive_kl.reward_std_weight: 0.15
actor_adaptive_kl.grad_norm_weight: 0.10

```

Phase B — corrected LSTM MetaKLController

```

yaml

actor_adaptive_kl.enable: true
actor_adaptive_kl.mode: lstm
actor_adaptive_kl.init_beta: 0.0001          # FIXED: was 0.001
actor_adaptive_kl.min_beta: 0.00001
actor_adaptive_kl.max_beta: 0.005
actor_adaptive_kl.warmup_steps: 30           # FIXED: was 150
actor_adaptive_kl.update_interval: 1         # FIXED: consistent
actor_adaptive_kl.ema_alpha: 0.90           # FIXED: consistent
actor_adaptive_kl.state_features:
  - kl_loss
  - reward_mean
  - reward_std
  - grad_norm
actor_adaptive_kl.lstm_hidden_dim: 32
actor_adaptive_kl.lstm_num_layers: 1
actor_adaptive_kl.mlp_hidden_dim: 32
actor_adaptive_kl.beta_output_activation: sigmoid_scaled
actor_adaptive_kl.lstm_reset_on_epoch: true  # NEW: explicit reset policy
actor_adaptive_kl.lstm_gradient_flow: true  # NEW: explicit gradient flow

```

2.2 Revised file plan

New files

`verl/trainer/ppo/actor_kl_controller.py` — Phase A rule-based controller

```
python
```

```

import torch

class RuleBasedActorKLController:
    """
    Phase A: rule-based adaptive beta_t.
    Increases beta when kl_loss > target band.
    Decreases beta when kl_loss < target band.
    """
    def __init__(self, cfg):
        self.beta = cfg.init_beta
        self.min_beta = cfg.min_beta
        self.max_beta = cfg.max_beta
        self.target = cfg.target_kl_loss # 0.003 - not 0.08
        self.up = cfg.up_gain
        self.down = cfg.down_gain
        self.alpha = cfg.ema_alpha
        self.warmup = cfg.warmup_steps
        self.interval = cfg.update_interval
        self.step = 0
        self._beta_ema = cfg.init_beta

    def update(self, state: dict) -> float:
        self.step += 1
        if self.step < self.warmup:
            return self._beta_ema
        if self.step % self.interval != 0:
            return self._beta_ema

        kl = state['kl_loss']
        raw = self.beta * (self.up if kl > self.target else self.down)
        raw = max(self.min_beta, min(self.max_beta, raw))
        self._beta_ema = self.alpha * self._beta_ema + (1 - self.alpha) * raw
        self.beta = self._beta_ema
        return self._beta_ema

    def reset(self):
        self.step = 0
        self._beta_ema = self.beta

```

`verl/trainer/ppo/meta_kl_controller.py` — Phase B LSTM controller

python

```

import torch
import torch.nn as nn

class MetaKLController(nn.Module):
    """
    Phase B: LSTM-based adaptive beta_t predictor.

    IMPORTANT – gradient flow:
    Gradients flow through beta_t into LSTM weights via:
        loss = policy_loss + beta_t * kl_loss
    LSTM is updated by the same optimiser step as the actor.
    No separate training loop required.

    IMPORTANT – hidden state reset:
    Reset (h, c) to zeros at start of each epoch.
    Do NOT reset every step – this destroys temporal memory.
    """
    def __init__(self, cfg):
        super().__init__()
        self.input_size = 4 # [kl_loss, reward_mean, reward_std, grad_norm]
        self.hidden_size = cfg.lstm_hidden_dim
        self.min_beta = cfg.min_beta
        self.max_beta = cfg.max_beta
        self.warmup = cfg.warmup_steps
        self.alpha = cfg.ema_alpha
        self.step = 0
        self._beta_ema = cfg.init_beta

        self.lstm = nn.LSTMCell(self.input_size, self.hidden_size)
        self.head = nn.Linear(self.hidden_size, 1)

        # Running normalisation for state features
        self.register_buffer('running_mean', torch.zeros(self.input_size))
        self.register_buffer('running_var', torch.ones(self.input_size))
        self.register_buffer('count', torch.tensor(0.0))

        self.hidden = None # (h, c) – reset each epoch

    def reset_hidden(self):
        """Call at start of each epoch, NOT each step."""
        self.hidden = None
        self.step = 0

    def _normalise(self, x: torch.Tensor) -> torch.Tensor:
        """Online running normalisation. Clip to [-5, 5]."""
        self.count += 1

```

```

delta = x - self.running_mean
self.running_mean += delta / self.count
self.running_var = (self.running_var * (self.count - 1) +
                    delta * (x - self.running_mean)) / self.count
std = (self.running_var + 1e-6).sqrt()
return torch.clamp((x - self.running_mean) / std, -5.0, 5.0)

def forward(self, state: dict) -> torch.Tensor:
    """
    Returns beta_t as a scalar tensor.
    Gradient flows back through beta_t into LSTM weights.
    """
    self.step += 1

    x = torch.tensor([
        state['kl_loss'],
        state['reward_mean'],
        state['reward_std'],
        state['grad_norm'],
    ], dtype=torch.float32)

    x = self._normalise(x)

    if self.hidden is None:
        h = torch.zeros(1, self.hidden_size)
        c = torch.zeros(1, self.hidden_size)
    else:
        h, c = self.hidden

    h, c = self.lstm(x.unsqueeze(0), (h, c))
    self.hidden = (h.detach(), c.detach())
    # Detach hidden from graph – only beta_t carries gradient

    raw = self.head(h)
    beta_t = self.min_beta + (self.max_beta - self.min_beta) * torch.sigmoid(raw)

    if self.step < self.warmup:
        # During warmup, return init_beta as non-differentiable constant
        return torch.tensor(self._beta_ema)

    # EMA smoothing (optional, kept for stability)
    beta_val = beta_t.item()
    self._beta_ema = self.alpha * self._beta_ema + (1 - self.alpha) * beta_val

    return beta_t.squeeze()

def predict_beta(self, state: dict) -> float:

```

```
"""Convenience wrapper for logging – returns float."""
with torch.no_grad():
    return self.forward(state).item()
```

Modified files

verl/workers/actor/dp_actor.py — minimal change, 3 locations

```
python

# Location 1: imports (top of file)
from verl.trainer.ppo.meta_kl_controller import MetaKLController
from verl.trainer.ppo.actor_kl_controller import RuleBasedActorKLController

# Location 2: inside __init__
cfg = self.config.actor_adaptive_kl
if cfg.enable:
    if cfg.mode == 'lstm':
        self.kl_controller = MetaKLController(cfg).to(self.device)
    elif cfg.mode == 'rule':
        self.kl_controller = RuleBasedActorKLController(cfg)
    else:
        self.kl_controller = None
else:
    self.kl_controller = None

# Location 3: inside update_policy() – find kl_loss_coef and replace
# BEFORE:
#   loss = pg_loss + self.config.actor.kl_loss_coef * kl_loss

# AFTER:
if self.kl_controller is not None:
    state = {
        'kl_loss': kl_loss.item(),
        'reward_mean': rewards.mean().item(),
        'reward_std': rewards.std().item(),
        'grad_norm': grad_norm.item() if grad_norm is not None else 0.0,
    }
    beta_t = self.kl_controller(state)    # gradient flows here
    loss = pg_loss + beta_t * kl_loss
    metrics['actor/kl_coef_dynamic'] = float(beta_t)
    metrics['actor/kl_penalty_term'] = float(beta_t * kl_loss)
else:
    loss = pg_loss + self.config.actor.kl_loss_coef * kl_loss
```

`verl/trainer/ppo/ray_trainer.py` — epoch boundary only

```
python

# Add at the start of each epoch loop:
if hasattr(actor_worker, 'kl_controller') and actor_worker.kl_controller is not None:
    if hasattr(actor_worker.kl_controller, 'reset_hidden'):
        actor_worker.kl_controller.reset_hidden() # LSTM: reset per epoch
    elif hasattr(actor_worker.kl_controller, 'reset'):
        actor_worker.kl_controller.reset() # Rule: reset step counter
```

2.3 Corrected ablation matrix

Run all four conditions with **identical seeds and config**, only changing the β mechanism:

Condition	beta mechanism	Expected outcome
A — Fixed $\beta = 1 \times 10^{-4}$	Static scalar (your baseline)	29.5% (1 epoch), ~33% (3 epochs)
B — Fixed β grid	$\beta \in \{5 \times 10^{-5}, 1 \times 10^{-4}, 2 \times 10^{-4}\}$	Finds best fixed β — upper bound for fixed approach
C — Rule-based adaptive	Phase A controller	Should beat A, validates interface
D — LSTM MetaKLController	Phase B controller	Your thesis claim — should beat A, B, C

The thesis claim is: $D > A$ by a meaningful margin (target +1.5 to +3.0 pp).
Condition C gives you a strong ablation: if D barely beats C, the contribution is weaker.
If D clearly beats C, it proves the LSTM adds value beyond simple rule adaptation.

2.4 Corrected success criteria

Metric	Minimum to claim success	Strong result
MATH500 accuracy (3 epoch)	+1.0 pp over fixed β baseline	+2.0 pp
KL loss variance	Lower than fixed β	Notably lower
β trajectory	Non-trivial variation across steps	Correlates with reward changes
No collapse	No entropy crash in Phase B	Stable throughout

2.5 Implementation order — what to do first

Day 1: Fix the config errors (Section 6.1 and 6.2 corrections)
Confirm baseline `kl_loss_coef = 0.0001` in `run_training.sh`

Day 2: Implement `RuleBasedActorKLController` (Phase A — ~50 lines)
Minimal `dp_actor.py` change
Run 1-epoch smoke test with rule-based controller
Verify `beta_t` is varying (not stuck at min or max)

Day 3: Run Phase A full 3-epoch experiment
Compare to fixed- β baseline

Day 4: Implement `MetaKLController` (Phase B — ~80 lines)
Verify gradient flow by checking LSTM weights change after step 1

Day 5: Run Phase B full 3-epoch experiment
Generate beta trajectory plot for thesis Chapter 6

Part 3 — One-Line Summary for Each Error

Error	One-line fix
Wrong <code>init_beta</code> (0.001 $\rightarrow$ 0.0001)	Match your confirmed baseline config exactly
Wrong <code>target_kl_loss</code> (0.08 $\rightarrow$ 0.003)	Set to $3\times$ your observed KL loss of 0.001
Wrong <code>warmup_steps</code> (150 $\rightarrow$ 30)	Do not silence controller for 34% of training
Missing gradient flow statement	Gradients flow through <code>beta_t</code> into LSTM — same optimiser step
EMA alpha mismatch	Use 0.90 for both Phase A and Phase B
<code>update_interval</code> mismatch	Use 1 for both Phase A and Phase B
Missing hidden state reset policy	Reset (h,c) at epoch start, never at step start
<code>ray_trainer.py</code> injection risk	Keep controller in <code>dp_actor.py</code> , only reset at epoch boundary via trainer

Revised proposal prepared: 2026-04-18

For: Ruth Tamiru MSc thesis — Adaptive KL Regularization via Meta-Learning